

PAISE: PIM-Accelerated Inference Scheduling Engine for Transformer-based LLM

Hyojung Lee, Daehyeon Baek, Jimyoung Son, Jieun Choi, Kihyo Moon, and Minsung Jang

Cloud Research Team, Samsung SDS

{hj824.lee, dh04.baek, jm2.son, jen.choi, kihyo.moon, minsung.jang}@samsung.com

Abstract—Transformer-based Large Language Models (LLMs) demand significant computational and memory resources due to the autoregressive token generation in decoder blocks. In particular, the attention layer in LLM models has low arithmetic intensity but high memory traffic, thus requiring frequent updates to the KV matrices with each decoder iteration. As a result, LLM inference becomes memory bound, leading to increased latency. To address this, we introduce PAISE, a framework leveraging Processing-In-Memory (PIM) technology to offload memory-intensive tasks. PAISE employs GPU-PIM heterogeneous computing resources to optimize inference operations in transformer-based LLMs. The framework comprises (i) a scheduling algorithm that decides which operations to offload to PIM based on model configuration and PIM hardware specifications and (ii) an enhanced PIM kernel that performs transaction-wise interleave-batched GEMM (General Matrix Multiplication) operations, maximizing data throughput via data layout adjustments. We implemented PAISE on the GPT-2 and Llama2-7B models using an AMD MI100 GPU with HBM-PIM devices. Our evaluations show that offloading the attention layer to PIM reduces execution time by up to 48.3% compared to GPU-only inference, demonstrating PAISE's significant potential to enhance the efficiency of LLM inference, which could lead to faster and more efficient AI applications.

I. INTRODUCTION

Transformer-based Large Language Models (LLMs) are increasingly being implemented across various applications [4], [8], [25], [27]. In transformer-based LLM inferences, a decoder block, consisting of multiple fully connected layers and attention layers, as illustrated in Figure 1, is repeatedly iterated as many times as the number of decoder blocks determined by the model configuration to generate a single token. This iterative process continues until the final token is generated. Consequently, inference serving for LLM-based applications significantly consumes computing and memory resources.

In a decoder block, the Fully Connected (FC) and attention layers differ in the computations required and the resources consumed. The FC layer maintains high arithmetic intensity and supports batch processing of repetitive operations with reusable pre-trained weights. In contrast, the attention layer exhibits low arithmetic intensity but high memory traffic requirements [3], [31], [5] because it requires matrix-vector multiplication between a query vector and independent Key-Value (KV) cache matrices for each request to calculate attention scores. This disparity leads to performance degradation in terms of throughput and latency due to memory constraints.

To overcome the limitations of memory bandwidth for LLM inferences, Processing-In-Memory (PIM) can be adopted as a hardware accelerator. PIM integrates a Processing Unit (PU) near each memory bank, allowing it to scale computational

throughput and data bandwidth in proportion to the number of banks [9], [15], [16], [19]–[21]. By leveraging these characteristics, offloading memory-intensive operations to PIM can enhance the performance of LLM inference and reduce operational costs.

However, determining which operations to offload to PIM is not straightforward. Several challenges arise when leveraging GPU-PIM heterogeneous computing resources, particularly when utilizing DRAM-based HBM-PIM [21]. The order in which PIM and GPUs read data from memory differs, which requires the GPU to update the memory area of the PIM according to its reading protocols for offloading operations. In addition, PIM has an optimal size of memory areas that can be processed efficiently at once, determined by hardware specifications such as banks, bankgroups, and channels. Therefore, it is crucial to consider the hardware architectural data layout to maximize computational performance in PIM. Another challenge is the potential for DRAM command sequence violations caused by host processor interventions, which can harm the correctness of PIM calculations. While HBM-PIM [21] employs barriers to maintain command sequence integrity, this solution does not entirely mitigate the issues and causes latency overhead.

This work introduces **PAISE** (**PIM-Accelerated Inference Scheduling Engine**), a novel framework aimed at significantly enhancing the performance and efficiency of LLM inference by effectively utilizing PIM technology, which comprises (i) a scheduling algorithm for offloading inference operations to PIM, and (ii) a modified PIM kernel based on the HBM-PIM library [36], enabling the execution of each LLM inference layer on PIM. The scheduling algorithm determines when and which operations to offload to PIM, considering the memory intensity of operations in transformer-based LLM inference layers and the effective utilization of PIM hardware features.

We implement a Data Layout Adjustment (DLA) module in the modified PIM kernel to align with the PIM-specific data processing protocol. Specifically, we introduce an interleaving weight matrix update method and a transaction-wise interleave-batched GEMM (General Matrix Multiplication) operation for PIM, which provides maximum data throughput by avoiding DRAM command sequence violations of the host processor. This approach facilitates batch processing in the attention layer to compute attention scores and values, thanks to the interleaved updating of Key and Value cache matrices.

Finally, we implement PAISE on an AMD Instinct MI100 GPU [1] equipped with HBM-PIM (Aquabolt-XL) [21]. The scheduling algorithm determines which operations during the

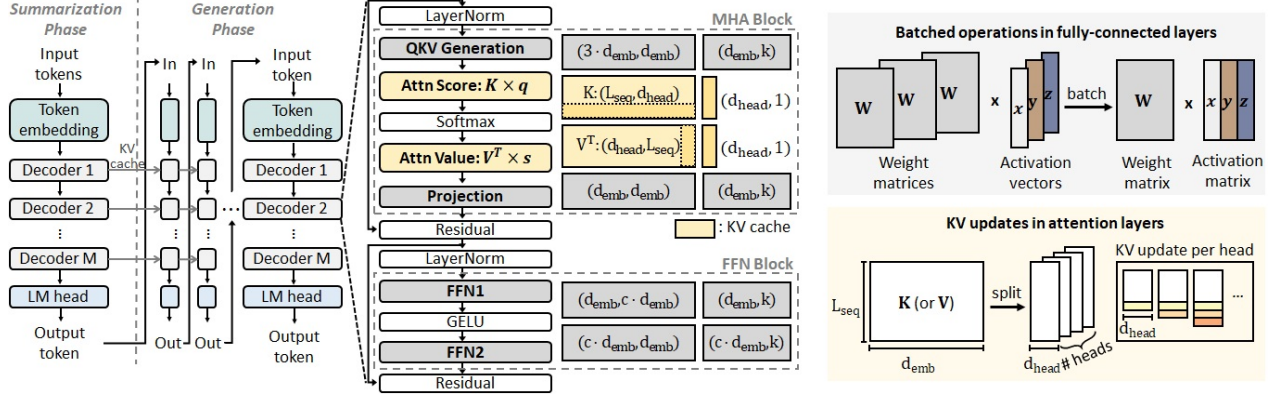


Fig. 1. Transformer-based LLM inference and operation layers within a decoder in the generation phase. FC layers and attention layers are colored in gray and yellow, respectively. The dimensions of matrices required for each operation are specified, where the following are denoted: embedding vector dimension d_{emb} , maximum sequence length L_{seq} , head dimension d_{head} , projection rate c , and batch size k .

GPT-2 [32] and Llama2 [37] inference process should be offloaded to PIM. For this purpose, we decompose each operational layer, as shown in Fig. 1, and propose methods for offloading to PIM that are adaptive to the characteristics of the FC layer and the attention score and value layers. Particularly for the attention layer, each decoder block operation requires updating the KV cache matrices in the PIM memory area, for which we suggest a bulk-memory copy module to minimize the overhead on the host processor. Our evaluation shows that offloading the attention layer to PIM reduces execution time by up to 48.3% compared to GPU-only inference. This result underscores the potential utility of PIM in LLM inference as PIM technology advances, promising a bright future for LLM technology.

- o To enable PIM in transformer-based LLM inference, we implement a DLA method that maximizes the data bandwidth of actual PIM devices and addresses the limitations arising from hardware characteristics and constraints imposed by the DRAM controller. Additionally, we develop a modified PIM kernel capable of performing operations for attention layers.
- o We propose the scheduling algorithm, which decomposes the process of transformer-based LLM inference [38] into individual computational layers and determines which operations to offload to PIM optimally. This strategy is based on inference serving configurations and hardware specifications of PIM, aiming to enhance throughput, reduce latency, and improve resource utilization efficiency.
- o Finally, using the GPT-2 and Llama2 models as examples, we demonstrate the specific application of the scheduling algorithm and how the PIM kernel is adapted for processing each layer of inference within PIM. Our evaluation based on this example illustrates the significant improvements in throughput, latency, and power consumption that the GPU-PIM heterogeneous framework, PAISE, achieves compared to GPUs.

II. BACKGROUND

In this section, we provide a comprehensive overview of the transformer-based LLM inference [38] process and the

hardware architecture of PIM, which together forms the basis of our PAISE framework.

A. Transformer-based LLM Inference

The transformer-based LLM inference [38] process consists of the summarization (or prefill) and generation (or decode) phases, as shown on the left side of Fig. 1. The first output token is generated based on the input sequence in the summarization phase. The generation phase, which PAISE focuses on, produces subsequent tokens using the previously generated token as input. This phase is resource-intensive due to its iterative nature, making it a prime candidate for optimization through PIM. A decoder in this context comprises a Multi-Head Attention (MHA) block and a Feed-Forward Network (FFN) block, as shown in the middle of Fig. 1. Each of these blocks is preceded by a normalization layer and followed by a residual connection layer. The MHA block begins with the QKV generation layer, where the query (Q), key (K), and value (V) matrices are calculated. The attention scores are computed as the inner product of the Q and K matrices, which are then normalized using a softmax function to produce attention weights. These weights are applied to the V matrix to compute the attention values. In this process, most LLMs store the K and V matrices in memory to reuse in subsequent generations to reduce the number of computations, called the KV caching technique. Finally, the FFN block consists of two layers, FFN1 and FFN2, separated by a GELU (Gaussian Error Linear Unit) activation function.

Among the layers within the decoder, as shown on the right side of Fig. 1, the FC and attention layers are colored gray and yellow, respectively. The FC and attention layers have different characteristics. On the one hand, during the generation phase, FC layers typically perform a General Matrix-Vector multiplication (GEMV) operation, which is the multiplication of a weight matrix based on the pre-trained weights of the model and an activation vector. However, when input prompts are batched for serving, the multiple GEMV operations can be transformed into a GEMM operation by concatenating the activation vectors into a matrix.

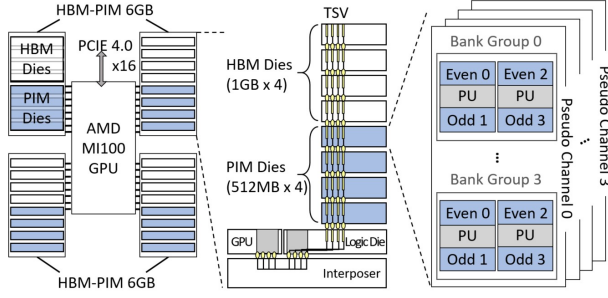


Fig. 2. GPU-PIM System-in-Package (SiP) architecture. The architecture includes the AMD MI100 (left), a 3D-stacked HBM-PIM (middle), and the hierarchical memory architecture within a PIM die (right).

On the other hand, the attention layers, Attention Score (AS) layer and Attention Value (AV) layer, perform activation-activation multiplications. The operand weight matrices of the layers, K and V matrices, are divided by the number of multi-heads, resulting in matrices with a width of d_{head} . In the summarization phase, the K and V matrices are initialized with key and value vectors, and the batch size is the number of input tokens. In the subsequent generation phase, these matrices expand with each token, reaching the maximum sequence length of L_{seq} . Therefore, a key characteristic of the attention layer is that it requires distinct K and V matrices for each token, leading to GEMV operations even during batched serving. This property leads to repeated GEMV operations during inference, causing the performance bottleneck of memory traffic, which motivated this work.

B. Performance Characteristics of FC and MHA Layers.

The FC and MHA layers in LLM inference behave differently in GEMM operations.

FC layers. For batch size k , weight height h , and width w , the FC layers execute a GEMM operation $W[h, w] \times I[k, w] = O[k, h]$, with batched input/output vectors I , O , and a fixed size weight W . In this case, the complexity of the number of computations is $\mathcal{O}(khw)$, but that of required memory I/O size is $\mathcal{O}(kh + hw + kw)$. As k is much smaller than h and w (i.e., $k \ll h, w$) for most cases in the generation phase, this would be simplified to $\mathcal{O}(hw)$. As the batch size k grows, the number of computations surpasses that of memory reads/writes, which results in compute-bound for large k . Still, with small batch size $k \simeq 1$, the number of computations and memory I/O is also similar, which behaves in a memory-bound manner.

MHA layers. MHA layers with the number of heads d_{head} execute a batched GEMV operation $W[k, d_{head}, h, w] \times I[k, d_{head}, w] = O[k, d_{head}, h]$, which is $k \times d_{head}$ number of batches of GEMV operations with $k \times d_{head}$ different weight matrices. In this case, the number of computations and memory I/O are asymptotically equivalent to $\mathcal{O}(kd_{head}hw)$. These two numbers are asymptotically equivalent regardless of the batch size k , so the MHA layers preserve memory-bound behavior for arbitrary batch sizes.

Comparison to other PIM accelerators. This characteristic is also explored in other simulator-based PIM accelerator studies such as NeuPIMs [10] and AttAcc [29]. However,

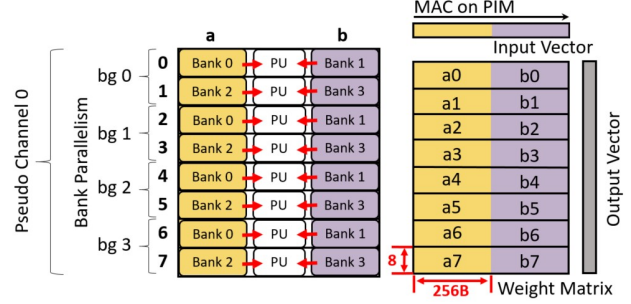


Fig. 3. Data layout in a pseudo channel. The data layout includes four bankgroups (bg), each comprising two bank pairs of even and odd banks.

what distinguishes PAISE is its operation on real hardware, specifically Aquabolt-XL [21]. In contrast, many previously proposed PIM-based accelerators [10], [17], [29], [40] depend on simulation environments that emulate PIM hardware. For example, NeuPIMs [10] and AttAcc [29] focus on optimizing PIM offloading for LLM inference through the PIM-GPU pipeline. Our work, however, takes a different approach by addressing the need for DLA in PIM and designing an offloading scheduling strategy. In this way, PAISE complements these existing studies and provides a synergistic contribution to the ongoing development of PIM technology.

C. Hardware Architecture of HBM-PIM Device

The left side of Fig. 2 shows the GPU-PIM System-in-Package (SiP) architecture based on the MI100 equipped with Aquabolt-XL [21], which includes four HBM-PIM stacks. Each stack is vertically stacked with four HBM dies and four PIM dies, communicating with the GPU via logic dies and an interposer by Through-Silicon Via (TSV). The GPU in this architecture plays a crucial role in data transfer to PIM, where direct inter-bank communication is impossible. Thus, memory transfer through the PHY interface is essential for heterogeneous computation between the GPU and PIM.

As shown in Fig. 2, the Aquabolt-XL has a hierarchical memory structure. Each SiP contains four chips, each with four PIM dies. Each PIM die includes four pseudo-channels, and each pseudo-channel consists of four bankgroups. Lastly, each bank group contains four banks (two even-odd bank pairs). This hierarchical structure results in a total of 1024 banks within a SiP. The adjacent PU can process 256B of weight matrix for each bank at one iteration inside the PIM kernel [21]. This corresponds to 128 elements for each bank, considering HBM-PIM supports FP16 precision. Each PU has two vector registers of $8 \times 32B$ and 16 FP16 adders and multipliers that can multiply and add 16 elements per cycle. This leads to specific conditions of the weight matrix size for GEMV operations that can be efficiently performed in PIM, which will be discussed in the following section.

Additionally, due to the limitation of PIM hardware, it cannot perform inference operations such as Softmax, residual, GELU, or Layernorm, and thus these were excluded from our offloading targets in this work.

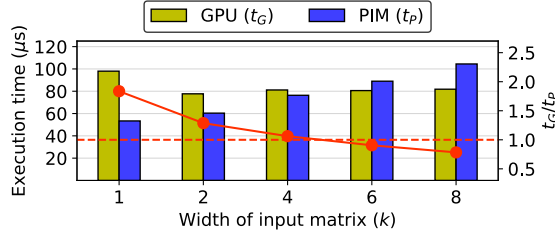


Fig. 4. The execution times for the GPU (t_G) and PIM (t_P) and their ratio (t_G/t_P) for GEMM operations. The weight matrix size is (4096, 256).

D. Data Layout and Bank-level Parallelism of PIM

In Fig. 3, the data layout in PIM illustrates how PIM operations are executed through bank-level parallelism. The PU conducts pairwise multiply-accumulate (MAC) operations, alternatively referencing even and odd banks. For example, a PU accesses neighboring even bank 0 (e.g., a0) and odd bank 1 (e.g., b0) sequentially. Each PU has a register size of 256B, resulting in the smallest width of a single operation across both even and odd banks being 512B or 256 elements in FP16 precision.

Considering (i) the pairwise MAC for even-odd banks, (ii) 512 bank-pairs in 64 pseudo channels in a GPU-PIM SiP, (iii) that a memory fence of DRAM commands [21] allows processing eight commands between the fences in a bank, and (iv) that a single DRAM command can be broadcast and processed by all PUs in parallel across within a pseudo channel, referred to as *all-bank* mode, a unit PIM weight matrix size is defined as (4096, 256). This matrix can be processed in PIM, minimizing the delay caused by the memory fence across all 512 bank pairs in 64 pseudo channels in a SiP.

This unique data layout and bank-level parallelism require a Data Layout Adjustment (DLA) method to optimize computations in PIM. This need has motivated our research into specific DLA methods and corresponding GEMV operations to accelerate LLM inference with PIM.

E. Performance of PIM-GEMM(V) Operations

Thanks to the bank-level parallelism, PIM devices offer significant advantages in throughput for GEMV operations compared to GPUs [3], [5], [31]. Fig. 4 presents our benchmark results from the MI100 with HBM-PIM [21], comparing the performance of standard PIM-GEMM(V) operations [35] to that of the GPU. The weight matrix size is the unit size, i.e., (4096, 256), and the k value in the x-axis is the width of the GEMM input matrix.¹ Denote t_G and t_P by the execution times of GPU and PIM, respectively.

The result in the figure clearly shows that PIM-GEMV, when $k = 1$, is effective compared to GPU, with the execution time ratio t_G/t_P exceeding 1.8. However, as the value k increases, the ratio decreases, indicating that the GPU becomes preferable for GEMM with larger k values. This performance decrease is due to how PIM concatenates GEMV operations to perform GEMM, whereas GPUs are optimized explicitly for GEMM operations. This emphasizes the performance benefits

¹Throughout this paper, GEMM operations are described as the multiplication of weight and input matrices, using the notation $(n, m) \times (m, k)$.

of PIM-GEMV operation under specific conditions, which is a key focus of our research. However, as the HBM-PIM device executes the PIM-GEMV kernel with hardware-specific restrictions, our primary strategies to leverage these benefits are as follows:

- **Efficient utilization of PIM:** PIM-GEMV kernel can maximize its computational bandwidth by processing a unit weight matrix of size (4096, 256) or multiples of this size. However, there are numerous kinds of weight matrix configurations, which include much smaller dimensions. Therefore, putting multiple small matrices into a (4096, 256) weight matrix is necessary, reducing zero padding as much as possible.
- **Avoiding DRAM command reordering:** DRAM controllers reorder instructions to optimize performance [34], but this reordering can disrupt the sequence of DRAM commands for PIM operations. Therefore, we aim to find DLA methods and GEMV techniques for PIM that avoid such reordering to maximize its computational efficiency.

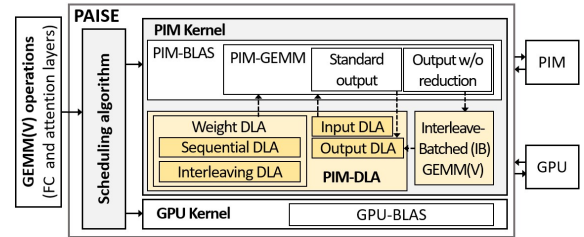


Fig. 5. Components of PAISE, including the scheduling algorithm, PIM-DLA, and IB-GEMM(V) modules. Solid arrows indicate the execution flow, while dashed arrows represent the data flow.

III. PAISE OVERVIEW

This section introduces PAISE, outlining the rationale behind its development and providing an overview of its structure. PAISE is designed to enhance the performance of transformer-based LLM inference by utilizing both PIM and GPU heterogeneous computing resources.

A. Rationale

The fundamental goal of PAISE is to improve the performance and resource efficiency of transformer-based LLM inference by offloading specific operations to PIM. PAISE consists of two main components:

- **Scheduling algorithm:** This scheduling algorithm determines which operations to offload to PIM. It leverages that PIM can outperform GPUs in memory-intensive GEMV and GEMM operations with a thin input matrix. The algorithm maximizes throughput and resource utilization by considering the configurations of LLM inference and PIM hardware specifications.
- **Software framework:** The framework, based on the HBM-PIM library [36], enables each layer of the LLM inference model to be executed on PIM. It includes optimal PIM memory assignment methods for weight matrices in GEMM operations, maximizing PIM performance without triggering its limitations. Additionally, it provides a modified PIM

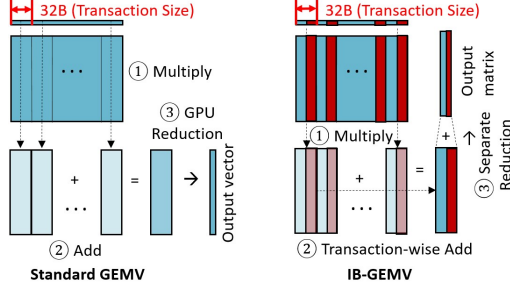


Fig. 6. PIM-GEMV operations.

kernel for multi-head attention layers, specifically for calculating attention scores and values.

B. Components of PAISE

Figure 5 illustrates the main building blocks for PAISE. The yellow-colored areas represent the software components in PAISE, which we will discuss in detail in Section IV. The remaining components are provided by the HBM-PIM library [36] and the GPU. The PAISE schedules GEMM operations in the FC and attention layers for offloading to PIM. Once the scheduling is determined, the PIM Data Layout Adjustment (PIM-DLA) module in the PIM kernel transforms the weight, input, and output matrices into data layouts that PIM can process.

The PIM-DLA module includes two distinct methods for handling weight matrices: sequential DLA and interleaving DLA. Sequential DLA allocates the weight matrix to banks either row-wise or column-wise, making it suitable for standard GEMM operations in PIM. In contrast, interleaving DLA minimizes idle banks and prevents disruptions to DRAM command reordering during GEMM operations. The interleave-batched (IB) GEMM module executes GEMM operations using the interleaving DLA, tailored for transaction-wise interleave-batched GEMM without GPU reduction. The specifics of this modified GEMM module will be detailed in the following section. Additionally, the input and output matrices for GEMM operations also require data transformation through DLA to be processed effectively in PIM.

IV. PAISE ARCHITECTURE

This section provides a detailed description of each PAISE component. First, we introduce the GEMV operations and corresponding DLA methods. Next, we present a scheduling algorithm that uses model configurations and PIM hardware specifications to determine which operations should be offloaded to PIM to enhance LLM inference performance. We then explain PAISE’s software architecture, detailing how PIM offloading is implemented. Finally, we describe the decision-making process for PIM offloading in the GPT-2 and Llama2 models.

A. Standard PIM-GEMV Operations

The left figure of Fig. 6 illustrates how GEMV operations are executed by the standard PIM kernel [35]. Initially, the input vector and weight matrix multiplication occur in 32 B transaction size units, which is a DRAM burst read/write

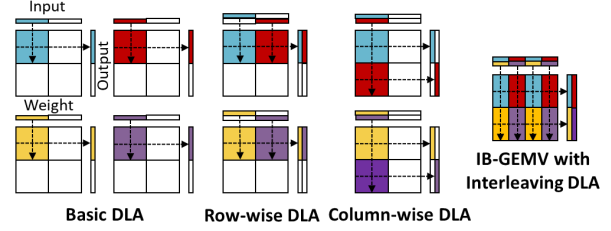


Fig. 7. Comparison of PIM-GEMV operations using different DLAs. The rightmost matrix uses IB-GEMV, while the others use standard GEMV.

length (8 burst length $\times$ 32 bit in this hardware). Then, the resulting matrices from this multiplication are accumulated, and the output vector is obtained through GPU reduction. However, this standard GEMV operation has limitations regarding memory throughput for arbitrary GEMV operations. Below, we explain two primary reasons for constraints:

- **Operand weight matrix size:** As discussed in Section II-D, the PIM device is optimized for a weight matrix size (4096, 256), that utilizes all banks and maximizes computational throughput based on its hardware specifications. However, operand weight matrices in practical GEMV operations often do not match this size, resulting in underutilization of the banks in all-bank mode.
- **GPU reduction:** Enhancing the efficiency of PIM-GEMV by placing multiple small operand matrices simultaneously in a (4096, 256) region is challenging because the PIM-GEMV kernel performs vector addition (matrix reduction by GPU) to obtain the output vector. This GEMV scheme makes distinguishing between the separate output vectors corresponding to multiple GEMVs within the final output impossible.

The left-far illustration in Fig. 7 shows the inefficiency of the standard GEMV operation with basic DLA, where the basic DLA arranges the weight matrix into the PIM area separately. The figure shows four standard GEMV operations (blue, red, yellow, and purple) that require four independent PIM weight matrices. For computational correctness, the GPU must fill zeros in the idle banks in each PIM unit matrix.

Sequential DLAs. To improve PIM utilization without modifying the standard PIM-GEMV kernel, we introduce two naïve approaches, row-wise and column-wise sequential DLAs.

- **Row-wise sequential DLA:** This method arranges weight matrices for GEMV operations in a row-wise sequence, ensuring the results of GEMV operations do not mix. The second figure in Fig. 7 demonstrates arranging each weight matrix for four GEMV operations in a row-wise sequence. The input vectors for each GEMV operation are placed to prevent the results of GEMV operations from being mixed.
- **Column-wise sequential DLA:** This method arranges weight matrices in a column-wise sequence, offering intuitive data arrangement for input vectors. The third illustration in Fig. 7 depicts arranging each weight matrix in a column-wise sequence. The input vectors are arranged to facilitate independent GEMV operations like the row-wise method.

Algorithm 1 PAISE Scheduling Algorithm

Input: Hidden embedding dimension d_{emb} , head dimension d_{head} , number of multi-heads h , number of decoders d , input sequence length s , maximum sequence length L_{seq} , batch size k , projection rate c

Output: Target resource r (GPU or PIM)

Parameter : Result weight matrix of DLA in PIM (n, m) ,
activated PIM memory area (n^*, m^*) ,
effectiveness of PIM α_k ,
idle banks inefficiency β ,
DLA and data movement overhead γ ,
offloading incentive u

```
1 initialize  $r \leftarrow \text{GPU}$ ,  $u \leftarrow 0$ ,  $n \leftarrow 1$ ,  $m \leftarrow 1$ ;  
   if Fully-connected layer then  
       /* FC layers */  
2       if QKV generation layer then  
3            $(n, m) \leftarrow \text{DLA}(3 \cdot d_{emb}, d_{emb})$ ;  
4       else if Projection layer then  
5            $(n, m) \leftarrow \text{DLA}(d_{emb}, d_{emb})$ ;  
6       else if FFN-1 layer then  
7            $(n, m) \leftarrow \text{DLA}(c \cdot d_{emb}, d_{emb})$ ;  
8       else  
9            $(n, m) \leftarrow \text{DLA}(d_{emb}, c \cdot d_{emb})$ ;  
10      end  
       /* Offloading incentive of FC layer */  
11       $u \leftarrow \alpha_k \cdot (L_{seq} - s) d n m - \gamma n m - \beta(n^* m^* - n m)$ ;  
12  else  
       /* Attention layers */  
13      if Attention Score layer then  
          /* AS layer */  
14          for heads in  $h$  do  
15               $(n, m) \leftarrow \text{DLA}((L_{seq}, d_{head}))$ ;  
16          end  
17      else  
          /* AV layer */  
18          for heads in  $h$  do  
19               $(n, m) \leftarrow \text{DLA}(d_{head}, L_{seq})$ ;  
20          end  
21      end  
       /* Offloading incentive of attn. layer */  
22       $u \leftarrow \alpha_1 \cdot (L_{seq} - s) d h \frac{n m}{2} - \gamma \frac{n m}{2} - \beta(n^* m^* - \frac{n m}{2})$ ;  
23  end  
       /* Offloading decision */  
24  if  $u > 0$  then  
25       $r \leftarrow \text{PIM}$ ;  
26  end
```

B. Interleave-Batched (IB) PIM-GEMV Operations

The IB-GEMV operation is similar to the standard GEMV, where a transaction-wise sum by a 32 B width is performed. However, it differs in forming an output matrix, reducing the number of steps, or removing the GPU reduction stage, as shown in the right side of Fig. 6. Doing so allows us to utilize the output matrix as *interleave-batched* output vectors from multiple GEMV operations. For example, two different colored (blue and red) operand weights are interleave-batched transaction-wise in the figure. After the MAC operations, the output matrix is obtained instead of the output vector of the standard GEMV. In this matrix, the output results of two different GEMVs (blue and red) are distinctly identifiable, from which we can drive the final output vectors of GEMVs through separate reduction. Our separate reduction approach accumulates the partial results from the same weight matrices

only for the final batched output vectors, which differs from the original version accumulates all 16 elements in a partial result transaction.

Interleaving DLA. This interleaving DLA method decomposes weight matrices from multiple GEMV operations and interleaves them into a PIM memory area. Each weight matrix is decomposed column-wise so that the width of the decomposed matrix fraction is a divisor of the transaction size, 32 B. Subsequently, the fractions are interleaved within a transaction size period. In the example shown in Fig. 7, four weight matrices are interleaved in a single PIM region thanks to interleaving DLA. The combination of this interleaving DLA method and the IB-GEMV operation can perform significantly better data throughput than the standard GEMV operation.

C. Scheduling Algorithm

The PAISE scheduling algorithm determines the optimal processing resource (PIM or GPU) for each computational layer in the decoder block based on model configurations and PIM hardware features.

- **Model configuration:** This includes hidden embedding dimension, number of multi-heads, head dimension, input sequence length, maximum sequence length, and batch size. These factors determine the size of operations and the reuse number of the weight matrix.
- **PIM hardware features:** These include the optimal size of the weight matrix, operational effectiveness of PIM in GEMM(V) operations, overhead from data layout adjustments, and inefficiency due to idle banks.

Policies of offloading layers. Before discussing the algorithm, it is essential to understand two key factors determining the feasibility of utilizing PIM. The first factor is the effectiveness α_k , which represents the performance advantage of PIM over GPU in GEMM operations with a k -width input matrix. For example, if $\alpha_k = 0.5$, PIM executes GEMM operations 50% faster than GPU. Thus, when $\alpha_k > 0$, PIM provides higher computational throughput than GPU, which makes it viable to consider PIM offloading.

The second factor is the reusability of the weight matrix. The offloading process includes two types of element-level overheads, γ and β . Nonetheless, it is possible to offset these overheads by reusing the weight matrix multiple times. The two types of overhead are as follows:

- The first overhead, γ , arises during DLA to align the weight matrix with PIM's communication protocol.
- The second overhead is memory inefficiency, β , which occurs when PIM's memory bandwidth is underutilized due to misalignment between the weight matrix size and PIM's optimal size. This memory inefficiency results in only partial bandwidth usage, with the rest of the banks padded with zeros.

These factors play a significant role in determining whether offloading is appropriate at different stages. In the summarization phase, where the weight matrix is reused less frequently, the overhead and inefficiency make offloading to PIM less advantageous. However, in the generation phase, where the weight matrix is reused repeatedly and the output length is

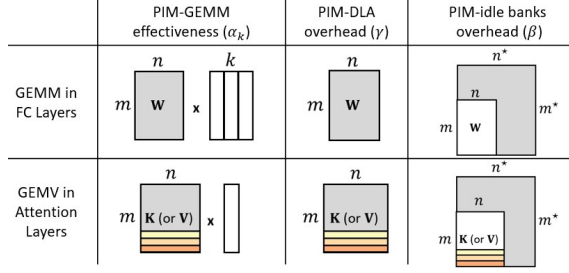


Fig. 8. Parameters affecting offloading strategies.

longer, offloading is more beneficial. Therefore, the PAISE scheduling algorithm focuses on optimizing offloading during the generation phase to fully leverage PIM's advantages.

Below is a detailed description of the offloading algorithm for the FC and attention layers during the generation phase.

Offloading strategy for FC layers. The FC layers, including QKV generation, projection, FFN1, and FFN2, require GEMV(M) operations based on the pre-trained weight matrix (see Fig.1). The gray-shaded area in Fig.8 illustrates where effectiveness and overheads occur at the element level.

- Impact of α_k : The weight matrix of size (n, m) is reused across iterations, where the number of iterations is determined by the number of decoders d and the output length $L_{seq} - s$. More iterations increase the incentive to use PIM.
- Impact of γ : The weight matrix is transformed into an (n, m) matrix via DLA and copied to the PIM memory area, incurring overhead γ .
- Impact of β : The inefficiency overhead caused by idle banks occurs in $n^*m^* - nm$ number of elements.

When the benefits of using PIM exceed the overhead, with $u > 0$ in line 11, GEMM operations are offloaded to PIM.

Offloading Strategy for Attention Layers. The offloading process for attention layers follows a similar pattern. K or V matrix is allocated in the PIM memory area (see Fig. 8).

- Impact of α_k : The efficiency of GEMV (i.e., $k = 1$) operations depends on α_1 , output tokens $L_{seq} - s$, decoders d , and heads h . The matrix is updated one row at a time, reducing reuse efficiency to half in average.
- Impact of γ : Each K matrix is transformed into an (n, m) matrix and copied to the PIM area, with the overhead γ halved due to row-wise updates.
- Impact of β : The idle-bank inefficiency overhead β occurs in $n^*m^* - nm/2$ number of elements.

If $u > 0$ in line 22, GEMV operations for the attention layers are offloaded to PIM.

D. Software Architecture of PAISE

To implement PAISE, we significantly modified the original PIM library code, open-sourced in Github, from Samsung Advanced Institute of Technology, written in HIP and OpenCL [33]. Our implementation of PAISE consists of a modified version of the C++ PIM Library and a Python wrapper for KV cache management, as depicted in Fig. 9. We introduced new GPU kernels to support our Interleaved-Batched GEMV kernel for weight transformation. In addition, we restructured the original GEMM kernel, which was initially launched as a

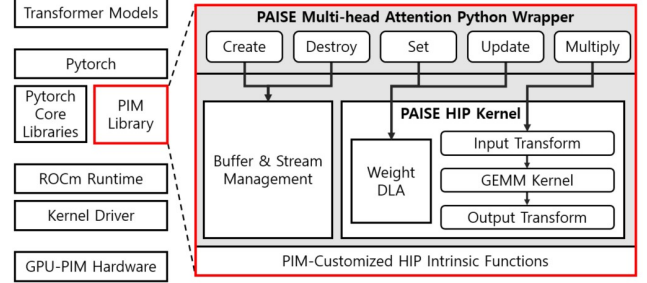


Fig. 9. Software architecture of PAISE.

single GPU kernel for computation, into three distinct steps: input transformation, actual PIM GEMM computation, and output transformation.

As illustrated in Fig. 9, we used the PIM buffer management code to allocate and free the KV cache region within the PIM's available physical memory region. After the allocation, we implemented four versions of weight DLA kernels: two for batched updates during KV cache initialization in the summarization phase and the others for single token updates in the subsequent generation phase. As the V cache should be stored as V^T , implementing K and V cache updates should differ, making two distinct kernel versions for each phase.

Once the KV cache is updated, PIM must perform the GEMM computation. The original code launches a single PIM kernel for GEMM computation. However, as seen in Fig. 6, the GPU must reorder the data layout of input and output vectors between logical representation and PIM-aware physical representation for maximum computational efficiency. Therefore, we divided the GEMM kernel into three steps: input transform, GEMM computation, and output transform. To minimize the memory allocation overhead of storing transformed inputs and intermediate outputs to be transformed, we pre-allocated enough size of input and output vector buffers within the PIM runtime. Given that the AS and KV kernels are invoked sequentially, sharing these input and output vector buffers across all attention kernels is feasible, thereby reducing memory usage.

In summary, PAISE optimizes the utilization of PIM in transformer-based LLM inference through strategic scheduling and data layout adjustments. PAISE aims to significantly enhance inference performance and resource efficiency by leveraging PIM's strengths and addressing its limitations.

V. EXAMPLE: LLM INFERENCE USING PAISE

This section details the execution flow for offloading operations to HBM-PIM [21] using the PAISE framework, exemplified by the GPT-2 [32] and Llama2-7B [37] model. Although these models are smaller than the recent advanced models such as Llama3 [23] and GPT-4 [28], their transformer architecture remains consistent with other models, which makes it suitable for validating the PIM offloading effects of transformer operations proposed in this paper. Therefore, this paper's offloading methodology and results can provide insights for applying PIM offloading to other models. We will now discuss how and which operations of these models can be offloaded to PIM.

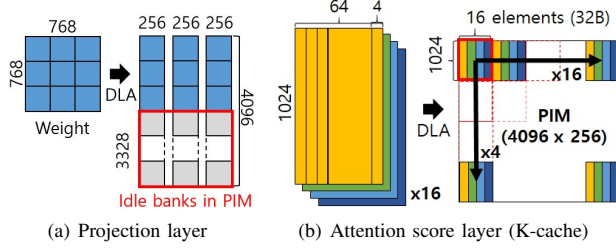


Fig. 10. Example of GPT-2: DLA of weight matrices for projection layer (768, 768) and attention score layer (1024, 64).

TABLE I
MODEL CONFIGURATIONS FOR BENCHMARKS

Model	d_{emb}	d_{head}	# head	# decoder	max. seq.
GPT-2 [32]	768	64	12	12	1024
Llama2-7B [37]	11008	128	32	32	2048

A. What to Offload

Upon evaluating the LLM models for PIM offloading, we identified the attention layers, the AS and AV layers, as ideal candidates, while the FC layers are conditionally suitable.

Not to offload all: FC layers. For the GPT-2 model, offloading the FC layers by updating the weight matrix in the PIM memory area via DLA leads to inefficiencies due to idle banks. In contrast, for the Llama2-7B model, while the weight matrices are appropriately sized for DLA without causing idle banks, the physical memory is insufficient. The specific details are as follows:

- **GPT-2:** Fig. 10(a) illustrates the DLA for the weight matrix of the GEMM operation in the projection layer² of the GPT-2 small model. The weight matrix, sized (768, 768), is divided and distributed across three PIM memory regions of size (4096, 256). During this process, idle banks occur in the PIM region with dimensions (3328, 768). To mitigate these overheads, the effectiveness of the GEMM operation α_k and the reuse rate of the weights must be sufficiently high. However, as the batch size increases, the effectiveness α_k decreases inversely, making it insufficient to overcome the overheads.
- **Llama2-7B:** The weight matrices of the FC layers consist of seven matrices: query, key, value, output projection matrices of (4096, 4096) dimensions, gate and up projection of (11008, 4096) dimensions, and down projection of (4096, 11008). These matrices are appropriately partitioned by DLA in the PIM region (4096, 256). Given the FP16 representation, storing all FC layers for the 32 decoder blocks of Llama2 requires over 12 GB of memory space, even with a batch size of 1. However, our PIM-GPU only provides 8 GB of physical memory for PIM operations, which is insufficient to store all FC layers. Thus, we can offload some FC layers to PIM only when the batch size is sufficiently small. For maximum PIM memory region use,

²The weight matrices of GEMM operation in other FC layers also experience idle bank inefficiency.

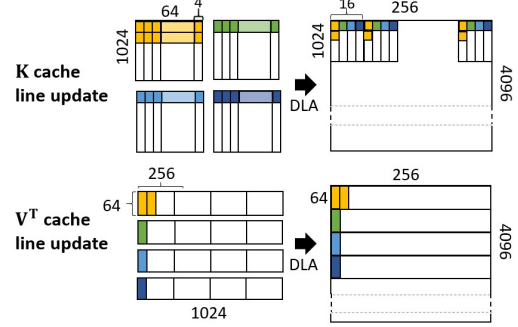


Fig. 11. Example of GPT-2: KV cache line updates.

TABLE II
END-TO-END LATENCY FOR LLAMA2-7B (SECONDS)

Batch size	GPU-only	PIM-GPU (Attn. only)	PIM-GPU (Attn. + FC (8 layers))
1	56.48	50.73	41.83
2	68.75	58.17	50.50
4	87.42	67.85	79.12

we choose output, up, and down projection matrices for PIM offloading with the memory space restriction. We choose down projection first, as it requires only 43 PIM regions: $(4096, 43 \times 256)$, smaller than 48 regions of the gate and up projections: $(3 \times 4096, 16 \times 256)$, with padding additional (1280, 4096).

To offload: Attention layers. In contrast, the attention layers of these models are well-suited for offloading to PIM for several reasons outlined below:

- **Interleaving DLA:** Using interleaving DLA, the unit weight matrix of size (4095, 256) can be efficiently updated in PIM without leaving any bank idle. Fig. 10(b) shows how the weight matrix for the GEMV operation in the AS layer of the GPT-2 small model is updated to the PIM memory area using interleaving DLA. For GPT-2, interleaving four distinct K cache matrices of size (1024, 64) into four columns produces partial weight matrices with a width of 256. Concatenating these interleaving partial weight matrices across the 16 memory channels results in a weight matrix that perfectly fits into the PIM area. In the case of Llama2, the matrix size is four times larger, with dimensions of (2048, 128). Therefore, it is simple to accommodate the weight matrices as a 2×2 PIM memory configuration.
- **Reusability of PIM memory:** The weight matrix generated through interleaving DLA has a high reuse rate due to its consolidation of multiple K cache matrices into a single PIM memory area.
- **Effectiveness of GEMV operation:** The computation for the attention score, $K \cdot Q^T$, is inherently a GEMV operation, ensuring that the operational effectiveness α_1 in PIM remains high.

A simple example of offloading. Table II presents the results of measuring the end-to-end latency of the Llama2-

7B model to validate the offloading decisions discussed. We use an input/output length of 2048/2048 and a maximum sequence length of 4096. When offloading the attention layers to PIM, inference throughput increases as the batch size grows compared to the GPU-only case. However, for the FC layers, only eight layers were offloaded to PIM due to the 8 GB PIM memory limitation. As the batch size increases, the effectiveness of offloading the FC layers diminishes, and at a batch size of 4, offloading the FC layers actually increases latency compared to offloading only the attention layers. This example demonstrates that offloading FC layers is beneficial only when the batch size is small.

B. How to Offload Attention Layers: Execution Flow

In this section, we explore the execution flow of how the PAISE framework offloads the attention layers of LLM models. As shown in Fig. 12, after QKV generation, the host processor issues a *Memcpy* command, which copies the update portions of the K cache and V cache corresponding to a single output token from GPU to PIM, allowing the updates to be concatenated with the existing K and V caches previously written in the PIM memory area.

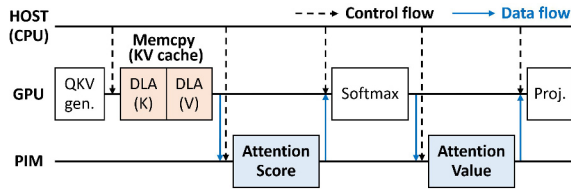


Fig. 12. Execution flow for offloading AS and AV layers to PIM.

Once the K and V caches are copied in PIM, the attention score and attention value can be calculated in AS and AV layers using the IB-GEMM kernel described in Section III-B. This execution flow is repeated throughout the generation phase, with the KV cache being concatenated as the output tokens increase. After the final token is generated, the PIM memory area is reset.

DLAs for KV caches of GPT-2. The GPT-2 small model has a maximum sequence length of 1024 and 12 heads, each with a dimension of 64. Thus, a single-head K cache is updated a row-wise with size (1024, 64) and V^T cache is updated a column-wise with size (64, 1024). As explained in Section V-A, when the batch size is 4, 48 K (or V) cache matrices (12 heads $\times$ 4 batches) can be efficiently packed into three optimal PIM memory areas without any idle banks since a unit PIM area (4096, 256) can accommodate 16 number of single-head K (or V) caches.

KV cache line updates of GPT-2. Fig. 11 precisely illustrates how the K and V caches are updated line by line in the PIM area. For K caches, according to the interleaving DLA rules described previously, each row of K cache (64 elements) is decomposed into four elements. Subsequently, four different K cache lines (yellow, green, blue, and navy) are interleaved to update the PIM area row-wise. This feature also applies to the V cache line updates since the V cache is updated column-wise (64, 1024), transferring new tokens to the PIM memory area with column-wise sequential DLA is relatively easy.

TABLE III
EVALUATION SYSTEM SETUP

Component	Description
CPU	2-socket AMD EPYC 7742 64 core processors. Hyperthreading disabled. Clock Speed: 2.25-3.4 Ghz.
Memory 2	DDR4 1TB 3200 MT/s
PCI-E	PCI-E 4.0 x16
GPU	AMD Instinct MI100 [1], up to FP16 184.6 TFLOPS. No Infinity Fabric Links between GPUs. Clock Speed: 1,200 Mhz.
PIM (VRAM)	Samsung Aquabolt-XL HBM2-PIM [16] 6GB. Clock Speed: 1,200 Mhz.
OS	Ubuntu 20.04.4 LTS, Linux 5.6.0-kfd+ x86_64.
Docker Environment	Host: Docker Engine Community 20.10.14. Container: Ubuntu 18.04.5 LTS.
Framework	AMD ROCm™ 4.0.0 [2] SAIT PIMLibrary [36] based PAISE.

ra	row	col_hi	bh	bg	bl	channel_hi	c1	ch	c0	offset					
34	33	20	19	17	16	15	14	13	12	8	7	6	5	4	0

Fig. 13. Physical Address Mapping of Aquabolt-XL.

VI. EVALUATION

In this section, we evaluate PAISE using the GPT-2 [32] and Llama2 [37] models on an AMD MI100 GPU [1] equipped with the HBM-PIM device [15], [16], [21]. We first describe our methodology and then analyze the impact of PAISE on performance and energy consumption.

A. Methodology

Our evaluation system setup is detailed in Table III. While the system has 8 AMD Instinct MI100 [1] GPUs, all benchmarks were tested using only one GPU. Table IV details the configuration of the Aquabolt-XL HBM-PIM [15], [16], [21] with the MI100 GPU. Fig. 13 illustrates the actual physical address mapping of the Aquabolt-XL HBM-PIM, with further details in Table V. We used the GPT-2 and Llama2 as the target, ensuring it operates stably within our hardware constraints while maintaining the characteristics of transformer-based LLMs. We utilized Hugging Face's GPT-2 [11] and Llama2 [12] models and pre-trained weights for inference. We assumed the input requests were sufficient in the queue, so we did not consider the arrival rate or queuing delay. Additionally, to apply PAISE to these models, we modified portions of the model code to allow using the IB-GEMM kernel through a Python-level API.

As a baseline, we assessed the performance of GPT-2 inference on the MI100 GPU without PIM, with ROCm 4.0, to ensure compatibility with Aquabolt-XL. Subsequently, we compared it to the performance of GPT-2 and Llama2 inference with PAISE, which offloaded attention layers to PIM, as discussed in Section V-A.

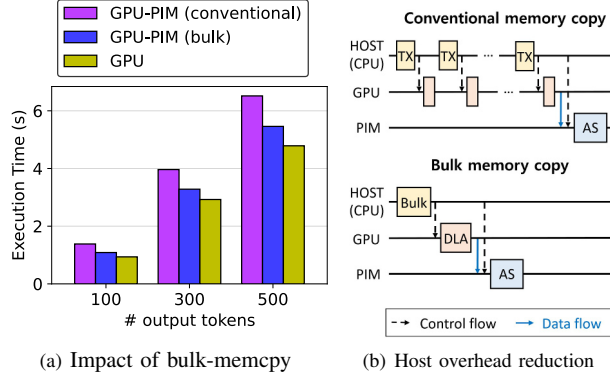


Fig. 14. Comparison end-to-end latency for GPT-2 model: GPU-only, GPU-PIM with conventional memcpy, and GPU-PIM with bulk memcpy. The input sequence length is 100. All values are averages from ten experiments.

TABLE IV
PIM CONFIGURATION

Component	Description	Component	Description
Technology	HBM2 [13]	Bank	4
Stacks	4 HBM 4 HBM-PIM	Row	16,384 for HBM 8,192 for PIM
Channel	16 per chip	Column	32
Rank	2	Burst Length	4
Bankgroup	4	Clock Speed	1,200 MHz

B. Performance: Impacts of PAISE

In this section, we evaluate the performance of PAISE from three perspectives. (1) First, we examine the limitations of the standard PIM-GEMV operation using the GPT-2 model to understand its performance constraints. (2) Second, we conduct a kernel-level analysis to evaluate the efficiency of interleaving DLA and IB-GEMV operations, focusing on both GPT-2 and Llama2 models. (3) Finally, we assess the improvements in end-to-end latency for GPT-2 when using PAISE.

(1) Limitations of Standard PIM-GEMV. We now present the limitations of offloading the AS layer by the sequential DLA of K cache and standard PIM-GEMV. We compare three cases: (i) GPU-only, (ii) GPU-PIM using conventional memory copy, and (iii) GPU-PIM using bulk-memory copy. With profiling using the ROC System Management Interface [2], the original implementation shows significant latency due to memory copies being performed in 32-byte transactions; thus, we developed a bulk-memory copy API to address this, simultaneously transferring K cache line updates from GPU to PIM, as shown in Fig. 14(b)). This reduces up to 21.4% of execution time over the conventional memory copy of the SAIT HBM-PIM library [36] (see Fig. 14(a)). However, the performance still lags behind the GPU due to the sequential DLA, which utilizes only 25% PIM memory bandwidth. As presented in the subsequent results, the interleaving update method effectively addresses these limitations.

(2) End-to-end Performance. Fig. 15 compares the latency of three cases using the interleaving DLA and IB-GEMV for offloading attention layers: (i) GPU-only, (ii) GPU-PIM with

TABLE V
PHYSICAL ADDRESS MAPPING OF AQUABOLT-XL

Component	Abbreviation	Bit	Description
Rank	ra	1	0 for PIM, 1 for HBM
Channel	channel_hi, ch	6	a pseudo-channel.
Bankgroup	bg	2	a bankgroup in a pseudo-channel
Bank	bh, bl	2	a bank in a bankgroup
Row	row	14	the MSE bit is 0 for the PIM region
Column	col_hi, cl, c0	5	a 32-byte transaction data in a 1KB memory row.
Offset	offset	5	not used for address mapping

AS layer (K cache) offloading, and (iii) GPU-PIM with AS and AV layers (KV cache) offloading, as indicated by circle, square and triangle markers. Fig. 15(a) shows the end-to-end latency for inference when output tokens exceed 200. The latency increases step-wise with the output length, and the lines represent linear regressions of three cases. This demonstrates that offloading either the K cache alone or both the K and V caches to PIM reduces latency by approximately 10% compared to a GPU-only case.

In Fig. 15(b), the Time Per Output Token (TPOT) graph exhibits a sawtooth pattern. This pattern results from the GPU increasing the memory required for tensor operations step-wise. Offloading the attention layers to PIM decreases the peak points of TPOT and lengthens the step periods. Figure 14(c) shows the step-wise increase in latency for each case when the number of output tokens is under 200. During this regime, the differences in latency between each case are not pronounced because the efficiency of PIM is not significant, as explained in Section V-C, when the number of iterations is small due to a short output length.

Fig. 15(d) shows the throughput enhancement from PIM offloading through PAISE. We calculated the end-to-end latency reduction for offloading cases relative to the GPU-only case. Since the effects of PIM appear step-wise, the overall throughput enhancement tends to fluctuate. Similar to previous interpretations, when the output length is low, the impact of PIM is unstable, leading to a tendency in which offloading to PIM increases the latency for some intervals under 200. However, because the step size is relatively large compared to the overall latency, a maximum performance improvement of 48.3% is observed at an output of 80 tokens. Meanwhile, when the output length exceeds 200 tokens, the performance gains from PIM become more stable overall.

Limitations of offloading attention value layer. In Figure 15, within some intervals, offloading only the AS layer to PIM based on the K cache results in shorter latency compared to offloading both the K and V caches and both the AS layer and AV layer to PIM. This is because the offloading techniques require additional data transformation overhead between PIM and GPU, which increases with the number of output tokens. Currently, PIM devices cannot execute their operations with GPU kernels simultaneously. In addition, for implementation

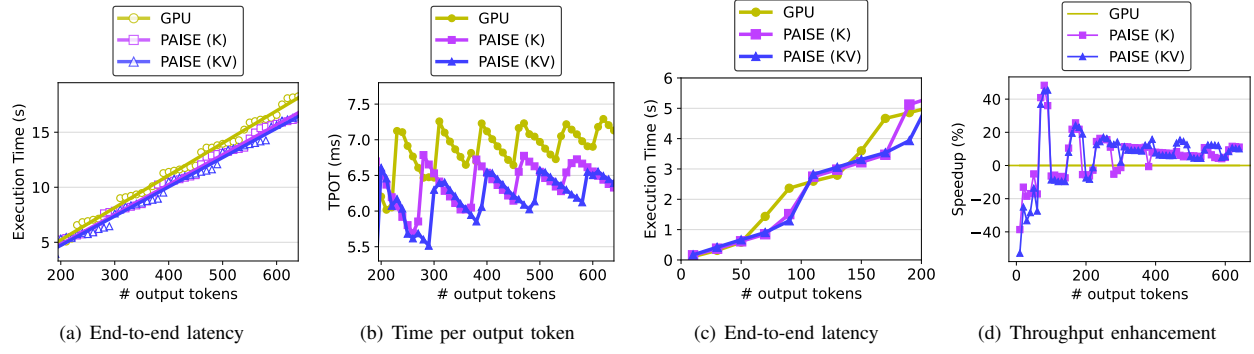


Fig. 15. Impacts of offloading AS layer (K cache) and AV layer (V cache) of GPT-2 model using IB-GEMV with interleaving DLA. The input length is 100, and output token length varies from 10 to 660. All values are averaged values over ten experiments.

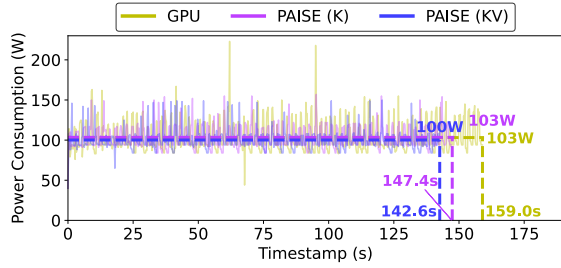


Fig. 16. Power consumption of the GPT-2 model with input and output token lengths of 100 and 660, respectively.

specifics, our PIM offloading technique introduces an additional HIP kernel stream for managing PIM kernel queues. While the original ROCm framework [2] uses a default HIP kernel stream to manage the GPU kernel management on LLM inferences, the MI100 GPU shows additional CPU overhead from switching GPU and PIM kernel streams.

(3) Input/Output Transformation Overhead. Table VI shows the breakdown of PIM-GEMM kernel execution stages on three models and batch size configurations. In this experiment, PAISE does not offload any FC layers of GPT2-medium and OPT-2.7B. In contrast, PAISE offloaded output, up, and down projections of Llama2-7B. The results say that the transformation overhead on small-batching FC layers is minuscule compared to the main PIM kernel, as the input/output vectors are relatively small and do not require complex index calculations for transformations. However, the output transformation overhead on the AS layer is pretty high compared to the AV and FC layers, especially on large batches. This is because the batched output of AS layer is larger than that of AV layer, where the sequence length is generally larger than the head dimension. In addition, the AS layers use Interleaving-DLA due to their smaller input and larger output, which requires some complex index calculations for output transformation. Other than that, this activation transformation overhead is acceptable.

(4) Layer Level Decomposition Figure 17 shows the layer-level kernel decomposition results that our IB-GEMV kernel implementation becomes more efficient on large-batching attention layers, but its efficiency diminishes on small-batching.

TABLE VI
INPUT/OUTPUT TRANSFORMATION OVERHEAD

Layer	In. Trans.	PIM-GEMM	Out. Trans.	Overhead
GPT2-Medium Batch 32				
$K \times q$ (AS)	20 μ s	177 μ s	68 μ s	33.2%
$V^T \times s$ (AV)	26 μ s	122 μ s	13 μ s	24.2%
OPT-2.7B Batch 4				
$K \times q$ (AS)	17 μ s	124 μ s	110 μ s	50.6%
$V^T \times s$ (AV)	25 μ s	111 μ s	13 μ s	25.5%
Llama2 Batch 1				
$K \times q$ (AS)	15 μ s	92 μ s	45 μ s	39.5%
$V^T \times s$ (AV)	25 μ s	61 μ s	7 μ s	34.4%
O Proj. (FC)	8 μ s	60 μ s	7 μ s	20.0%
Up Proj. (FC)	9 μ s	162 μ s	7 μ s	9.0%
Down Proj. (FC)	9 μ s	145 μ s	5 μ s	8.8%

This is because the KV cache matrix size depends on the batch size. For small batching cases, running attention layers on PIM is not effective enough compared to GPU-only execution, but in batch 32 cases, PIM shows 98.3% performance boost over GPU-only execution. For no batching FC layers, our IB-GEMV kernel outperforms GPU-only execution for 68.7% on PIM-offloaded FC layers of Llama2. However, these improvements are distilled due to the memory constraints as other FC layers are not offloaded to PIM.

In addition, while updating KV cache in GPU requires longer execution time as the output sequence grows, *our PIM-aware KV cache update kernels show constant time of fast token concatenation, with token-level direct memory write to PIM regions without reading KV cache matrices for concatenation.* However, for GEMV performance of PIM-offloaded attention layers could be lower than GPU-only execution on smaller token sequence index. This is because our implementation assumes maximum KV cache size for every token generation, but GPUs compute the same layer with smaller KV cache on smaller token sequence index.

C. Energy Consumption Analysis

With the reduced memory footprint of multi-head attention layer computation, we measured PAISE's energy efficiency

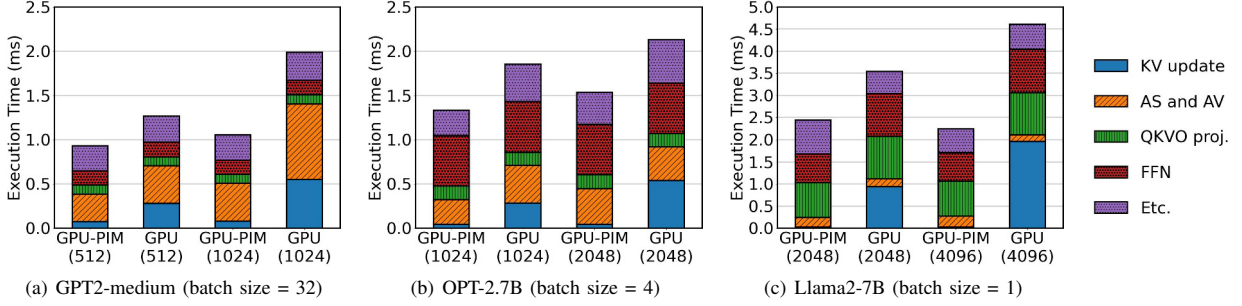


Fig. 17. Kernel level breakdown of a decoder block in generation phase, at mid and end point token sequences. (a) shows large batching on a small model, (b) is four batching on a medium-size model, and (c) a comparably large model with no batching.

over the baseline GPU-only scenario. For measurement, we used the ROC System Management Interface [2], which shows the power consumption of the specific timestamp of the entire GPU device, including the memory modules. With the power consumption numbers on these timestamps, we estimated the total energy consumption by multiplying the average power number (Watts) and the total execution time (seconds). Fig. 16 shows the measured real-time data with every 0.1s period. The average *power* consumption of the KV cache offloading case (100 W) is slightly lower than that of the GPU-only baseline (103 W). Still, the total *energy* consumption is reduced by around 11.5% with PAISE because the execution time to complete the inference tasks is reduced from 159.0s to 142.6s.

VII. RELATED WORKS

Transformer-based LLM, PIM, and PNM. In transformer-based LLMs, [38] attention layers require high memory bandwidth for data movements. Therefore, various studies have explored memory-efficient attention mechanisms for LLMs, offering scheduling and batching algorithms to enhance efficient inference serving systems or applying quantization and CUDA [26] kernel optimization to improve training and inference performance [6], [18], [39] to tackle the memory traffic requirements. On the hardware front, several memory devices designed for AI acceleration, such as PIM or PNM (Processing Near Memory), can enhance the performance of memory-intensive applications. PIMs, including HBM-PIM [21], Newton [9], and AESPA [14], achieve higher memory bandwidth utilization and more efficient energy usage than conventional systems. However, they entail higher development costs and complexities in SoC design validation. CXL-PNM [30] introduces a PNM architecture with LPDDR5X-based CXL memory to mitigate the drawbacks of PIMs.

PIM for LLM inference. Numerous PIM studies offer hardware-software co-design solutions for LLM inference, aiming to alleviate performance degradation caused by memory bandwidth constraints. For instance, NeuPIMs [10] and AttAcc [29] facilitate separate computation of operations in computing units (e.g., CPU, GPU, NPU) and PIMs. TransPIM [40] proposes a hybrid architecture featuring in-bank and near-bank computing units that exploit parallelism and support complex operations such as reduction and softmax. An optimal model partitioning algorithm [17] identifies which

operations should be offloaded to PIM devices through low-overhead profiling with ONNX runtime [24]. While previous studies typically conduct performance tests using DRAM simulators like DRAMsim3 [22], our performance results are based on actual PIM devices. Additionally, PAISE considers model architectures and PIM device characteristics, providing a unified general condition to determine if an operation should be offloaded to PIM.

VIII. FUTURE WORK

The modified GPU driver for PAISE does not support multi-process services due to the lack of fine-grained scheduling capability between multiple GPU-only kernels and PIM kernels. We currently hand-coded PIM kernels to ensure that no other kernels are executed simultaneously. Still, it is necessary to support a PIM-aware kernel scheduling to ensure the computational correctness and stability of the system. In addition, the current implementation of the PIM region memory allocator assumes a single process environment only for each GPU, which may cause issues related to virtual address translation. Therefore, we will develop a new PIM-aware page allocation strategy for implementing multi-process PIM kernel services.

Furthermore, expanding PIM-based inference acceleration to multi-GPU environments is essential for future work. This expansion will require addressing two main challenges: (i) implementing remote memory access across different GPUs at the kernel level for efficient data movement between GPUs and PIM, and (ii) ensuring compatibility with existing multi-GPU inference frameworks like DeepSpeed [33] and FlashAttention [7] to make full PIM integration. Tackling these challenges will enable PAISE to scale effectively across multi-GPU environments, improving performance and resource utilization.

IX. CONCLUSION

This work highlights how PIM can improve the inference performance of transformer-based LLMs and reduce computational costs by effectively using the PAISE framework for offloading memory-intensive operations. The enhancements in computational throughput and energy efficiency demonstrate PIM integration's practical advantages and future potential in LLM deployments.

REFERENCES

- [1] Advanced Micro Devices, Inc., “AMD CDNA Architecture,” <https://www.amd.com/content/dam/amd/en/documents/instinct-business-docs/white-papers/amd-cdna-white-paper.pdf>, 2020.
- [2] Advanced Micro Devices, Inc., “AMD ROCm™ Software,” <https://www.amd.com/en/products/software/rocm.html>, 2020.
- [3] J. Alsop, S. Aga, M. Ibrahim, M. Islam, A. McCrabb, and N. Jayasena, “Inclusive-pim: Hardware-software co-design for broad acceleration on commercial pim architectures,” *arXiv preprint arXiv:2309.07984*, 2023.
- [4] Anthropic, “Claude,” <https://www.anthropic.com/claude>, 2024.
- [5] B. Y. Cho, J. Jung, and M. Erez, “Accelerating bandwidth-bound deep learning inference with main-memory accelerators,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2021.
- [6] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, “Flashattention: Fast and memory-efficient exact attention with io-awareness,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 16 344–16 359, 2022.
- [7] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, “Flashattention: Fast and memory-efficient exact attention with io-awareness,” 2022. [Online]. Available: <https://arxiv.org/abs/2205.14135>
- [8] Google, “Gemini: A family of highly capable multimodal models,” 2024. [Online]. Available: <https://arxiv.org/abs/2312.11805>
- [9] M. He, C. Song, I. Kim, C. Jeong, S. Kim, I. Park, M. Thottethodi, and T. Vijaykumar, “Newton: A dram-maker’s accelerator-in-memory (aim) architecture for machine learning,” in *Proceedings of 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2020.
- [10] G. Heo, S. Lee, J. Cho, H. Choi, S. Lee, H. Ham, G. Kim, D. Mahajan, and J. Park, “Neupims: Npu-pim heterogeneous acceleration for batched llm inferencing,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024.
- [11] Hugging Face, “GPT-2,” <https://huggingface.co/openai-community/gpt2>, 2024.
- [12] Hugging Face, “Llama2,” <https://huggingface.co/meta-llama/Llama-2-7b-hf>, 2024.
- [13] JEDEC, “High Bandwidth Memory (HBM) DRAM JESD235D,” <https://www.jedec.org/standards-documents/docs/jesd235a>, 2021.
- [14] H. Kal, C. Yoo, and W. W. Ro, “Aespa: Asynchronous execution scheme to exploit bank-level parallelism of processing-in-memory,” in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2023.
- [15] J. H. Kim, S.-H. Kang, S. Lee, H. Kim, Y. Ro, S. Lee, D. Wang, J. Choi, J. So, Y. Cho, J. Song, J. Cho, K. Sohn, and N. S. Kim, “Aquabolt-xl hbm2-pim, lpddr5-pim with in-memory processing, and axdimm with acceleration buffer,” *IEEE Micro*, vol. 42, no. 3, pp. 20–30, 2022.
- [16] J. H. Kim, S.-h. Kang, S. Lee, H. Kim, W. Song, Y. Ro, S. Lee, D. Wang, H. Shin, B. Phuah, J. Choi, J. So, Y. Cho, J. Song, J. Choi, J. Cho, K. Sohn, Y. Sohn, K. Park, and N. S. Kim, “Aquabolt-xl: Samsung hbm2-pim with in-memory processing for ml accelerators and beyond,” in *Proceedings of IEEE Hot Chips 33 Symposium (HCS)*, 2021.
- [17] S. Y. Kim, J. Lee, Y. Paik, C. H. Kim, W. J. Lee, and S. W. Kim, “Optimal model partitioning with low-overhead profiling on the pim-based platform for deep learning inference,” *ACM Transactions on Design Automation of Electronic Systems*, vol. 29, no. 2, pp. 1–22, 2024.
- [18] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with pagedattention,” in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.
- [19] Y. Kwon, K. Vladimir, N. Kim, W. Shin, J. Won, M. Lee, H. Joo, H. Choi, G. Kim, B. An *et al.*, “System architecture and software stack for gddr6-aim,” in *Proceedings of IEEE Hot Chips 34 Symposium (HCS)*, 2020.
- [20] S. Lee, K. Kim, S. Oh, J. Park, G. Hong, D. Ka, K. Hwang, J. Park, K. Kang, J. Kim *et al.*, “A 1ynm 1.25 v 8gb, 16gb/s/pin gddr6-based accelerator-in-memory supporting 1tflops mac operation and various activation functions for deep-learning applications,” in *Proceedings of IEEE International Solid-State Circuits Conference (ISSCC)*, 2022.
- [21] S. Lee, S.-h. Kang, J. Lee, H. Kim, E. Lee, S. Seo, H. Yoon, S. Lee, K. Lim, H. Shin, J. Kim, S. O, A. Iyer, D. Wang, K. Sohn, and N. S. Kim, “Hardware architecture and software stack for pim based on commercial dram technology: Industrial product,” in *Proceedings of IEEE/ACM 48th Annual International Symposium on Computer Architecture (ISCA)*, 2021.
- [22] S. Li, Z. Yang, D. Reddy, A. Srivastava, and B. Jacob, “Dramsim3: A cycle-accurate, thermal-capable dram simulator,” *IEEE Computer Architecture Letters*, vol. 19, no. 2, pp. 106–109, 2020.
- [23] Meta, “Llama3,” <https://llama.meta.com/llama3/>, 2024.
- [24] Microsoft Corporation, “ONNX Runtime,” <https://onnxruntime.ai/>, 2018.
- [25] Microsoft Corporation, “Github Copilot,” <https://github.com/features/copilot>, 2024.
- [26] NVIDIA Corporation, “CUDA Toolkit,” <https://developer.nvidia.com/cuda-toolkit>, 2007.
- [27] OpenAI, “ChatGPT,” <https://chat.openai.com>, 2024.
- [28] OpenAI, “Gpt-4 technical report,” 2024. [Online]. Available: <https://arxiv.org/abs/2303.08774>
- [29] J. Park, J. Choi, K. Kyung, M. J. Kim, Y. Kwon, N. S. Kim, and J. H. Ahn, “Attacc! unleashing the power of pim for batched transformer-based generative model inference,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024.
- [30] S.-S. Park, K. Kim, J. So, J. Jung, J. Lee, K. Woo, N. Kim, Y. Lee, H. Kim, Y. Kwon *et al.*, “An lpddr-based cxl-pnm platform for tco-efficient inference of transformer-based large language models,” in *Proceedings of IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2024.
- [31] S. Pati, S. Aga, N. Jayasena, and M. D. Sinclair, “Demystifying bert: System design implications,” in *Proceedings of IEEE International Symposium on Workload Characterization (IISWC)*, 2022.
- [32] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language models are unsupervised multitask learners,” 2019.
- [33] J. Rasley, S. Rajbhandari, O. Ruwase, and Y. He, “Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters,” in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, ser. KDD ’20. New York, NY, USA: Association for Computing Machinery, 2020, p. 3505–3506.
- [34] S. Rixner, W. J. Dally, U. J. Kapasi, P. Mattson, and J. D. Owens, “Memory access scheduling,” *ACM SIGARCH Computer Architecture News*, vol. 28, no. 2, pp. 128–138, 2000.
- [35] Samsung Advanced Institute of Technology, <https://github.com/SAITPublic/PIMLibrary>, 2023.
- [36] Samsung Advanced Institute of Technology (SAIT), “PIMLibrary,” <https://github.com/SAITPublic/PIMLibrary>, 2024.
- [37] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. C. Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V. Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P. S. Koura, M.-A. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom, “Llama 2: Open foundation and fine-tuned chat models,” 2023. [Online]. Available: <https://arxiv.org/abs/2307.09288>
- [38] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” 2023.
- [39] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, “Orca: A distributed serving system for {Transformer-Based} generative models,” in *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022.
- [40] M. Zhou, W. Xu, J. Kang, and T. Rosing, “Transpim: A memory-based acceleration via software-hardware co-design for transformer,” in *Proceedings of IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2022.